

You can connect your QML dashboard application to a physical potentiometer and Arduino/Raspberry Pi by creating a bridge between your PC application and the embedded system. Here's how to set it up:

Option 1: Arduino + Potentiometer Approach

Hardware Setup

1. Connect your potentiometer to an analog input pin on the Arduino
2. Connect the Arduino to your PC via USB

Software Architecture

1. **Arduino code:** Read analog values from the potentiometer
2. **Serial communication:** Send data from Arduino to PC
3. **QML integration:** Create a C++ class to read serial data and update your dashboard model

Implementation Steps

1. Arduino Code

cpp

```
const int potPin = A0; // Analog pin for potentiometer

void setup() {
  Serial.begin(9600); // Start serial communication at 9600 bps
}

void loop() {
  // Read the potentiometer value (0-1023)
  int potValue = analogRead(potPin);

  // Map it to your gauge's range (e.g., for speed 0-220)
```

```
int mappedValue = map(potValue, 0, 1023, 0, 220);

// Send the value over serial
Serial.print("S:"); // Prefix to identify this as speed data
Serial.println(mappedValue);

delay(100); // Small delay to avoid flooding the serial port
}
```

For multiple potentiometers, expand the code:

cpp

```
const int speedPotPin = A0;
const int fuelPotPin = A1;
const int tempPotPin = A2;
const int gearPotPin = A3;

void loop() {
    // Read and map each potentiometer
    int speedValue = map(analogRead(speedPotPin), 0, 1023, 0, 220);
    int fuelValue = map(analogRead(fuelPotPin), 0, 1023, 0, 100);
    int tempValue = map(analogRead(tempPotPin), 0, 1023, 50, 130);
    int gearValue = map(analogRead(gearPotPin), 0, 1023, 1, 6);

    // Send values with identifiers
    Serial.print("S:"); Serial.println(speedValue);
    Serial.print("F:"); Serial.println(fuelValue);
    Serial.print("T:"); Serial.println(tempValue);
    Serial.print("G:"); Serial.println(gearValue);

    delay(50);
}
```

2. C++ Serial Reader for QML

Create a `SerialReader` class in your Qt project:

cpp

```
// serialreader.h
#ifndef SERIALREADER_H
#define SERIALREADER_H

#include <QObject>
#include <QSerialPort>
#include <QTimer>
#include "dashboardmodel.h"

class SerialReader : public QObject
{
    Q_OBJECT

public:
    explicit SerialReader(DashboardModel* model, QObject *parent = nullptr)
        : SerialReader();

    Q_INVOKABLE bool connectToPort(const QString &portName);
    Q_INVOKABLE void disconnectFromPort();
    Q_INVOKABLE QStringList getAvailablePorts();

private slots:
    void readSerialData();
    void handleError(QSerialPort::SerialPortError error);

private:
    QSerialPort *m_serialPort;
    DashboardModel *m_dashboardModel;
    QString m_buffer;
};

#endif // SERIALREADER_H
```

cpp

```
// serialreader.cpp
#include "serialreader.h"
#include <QSerialPortInfo>
#include <QDebug>

SerialReader::SerialReader(DashboardModel* model, QObject *parent)
```

```
        : QObject(parent)
        , m_serialPort(new QSerialPort(this))
        , m_dashboardModel(model)
    {
        connect(m_serialPort, &QSerialPort::readyRead, this, &SerialReader::readData);
        connect(m_serialPort, &QSerialPort::errorOccurred, this, &SerialReader::onError);
    }
};
```

```
SerialReader::~SerialReader()
```

```
{
    if (m_serialPort->isOpen())
        m_serialPort->close();
}
```

```
bool SerialReader::connectToPort(const QString &portName)
```

```
{
    m_serialPort->setPortName(portName);
    m_serialPort->setBaudRate(QSerialPort::Baud9600);
    m_serialPort->setDataBits(QSerialPort::Data8);
    m_serialPort->setParity(QSerialPort::NoParity);
    m_serialPort->setStopBits(QSerialPort::OneStop);
    m_serialPort->setFlowControl(QSerialPort::NoFlowControl);

    if (m_serialPort->open(QIODevice::ReadOnly)) {
        qDebug() << "Connected to" << portName;
        return true;
    } else {
        qDebug() << "Failed to open port" << portName << ":" << m_serialPort->portName();
        return false;
    }
}
```

```
void SerialReader::disconnectFromPort()
```

```
{
    if (m_serialPort->isOpen()) {
        m_serialPort->close();
        qDebug() << "Disconnected from serial port";
    }
}
```

```
QStringList SerialReader::getAvailablePorts()
```

```
{
```

```
QStringList result;
const auto serialPortInfos = QSerialPortInfo::availablePorts();

for (const QSerialPortInfo &info : serialPortInfos) {
    // Filter out Bluetooth ports if needed
    if (!info.portName().contains("Bluetooth")) {
        result << info.portName();
    }
}

return result;
}

void SerialReader::readSerialData()
{
    QByteArray data = m_serialPort->readAll();
    m_buffer.append(QString::fromUtf8(data));

    // Process complete lines
    int newlinePos = m_buffer.indexOf('\n');
    while (newlinePos >= 0) {
        QString line = m_buffer.left(newlinePos).trimmed();
        m_buffer.remove(0, newlinePos + 1);

        // Process the line
        if (line.startsWith("S:")) {
            bool ok;
            int value = line.mid(2).toInt(&ok);
            if (ok) {
                m_dashboardModel->setSpeed(value);
            }
        } else if (line.startsWith("F:")) {
            bool ok;
            int value = line.mid(2).toInt(&ok);
            if (ok) {
                m_dashboardModel->setFuel(value);
            }
        } else if (line.startsWith("T:")) {
            bool ok;
            int value = line.mid(2).toInt(&ok);
            if (ok) {
                m_dashboardModel->setTemperature(value);
            }
        }
    }
}
```

```
    }
    } else if (line.startsWith("G:")) {
        bool ok;
        int value = line.mid(2).toInt(&ok);
        if (ok) {
            m_dashboardModel->setGear(value);
        }
    }

    newlinePos = m_buffer.indexOf('\n');
}
}

void SerialReader::handleError(QSerialPort::SerialPortError error)
{
    if (error == QSerialPort::ResourceError) {
        qDebug() << "Serial port error:" << m_serialPort->errorString();
        disconnectFromPort();
    }
}
```

3. Integrate SerialReader into Your Application

Update your main.cpp :

cpp

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
#include "dashboardmodel.h"
#include "serialreader.h"

int main(int argc, char *argv[])
{
    QCoreApplication::setAttribute(Qt::AA_EnableHighDpiScaling);
    QGuiApplication app(argc, argv);

    QQmlApplicationEngine engine;

    // Create and expose our dashboard model to QML
```

```
DashboardModel dashboardModel;
engine.rootContext()->setContextProperty("dashboardModel", &dashboardMo

// Create and expose serial reader
SerialReader serialReader(&dashboardModel);
engine.rootContext()->setContextProperty("serialReader", &serialReader)

const QUrl url(QStringLiteral("qrc:/QML/main.qml"));
QObject::connect(&engine, &QQmlApplicationEngine::objectCreated,
                &app, [url](QObject *obj, const QUrl &objUrl) {
    if (!obj && url == objUrl)
        QCoreApplication::exit(-1);
    }, Qt::QueuedConnection);
engine.load(url);
return app.exec();
}
```

4. Create a Serial Connection UI in QML

Add a serial port connection interface to your `main.qml`:

qml

```
// Add to imports
import QtQuick.Dialogs 1.3

// Add this component to your main.qml
Rectangle {
    id: serialControlPanel
    anchors {
        left: parent.left
        right: parent.right
        top: parent.top
        margins: 10
    }
    height: 40
    color: "#2a2a2a"
    radius: 8

    RowLayout {
        anchors.fill: parent
```

```
anchors.margins: 5
spacing: 10

Text {
    text: "Serial Port:"
    color: "white"
}

ComboBox {
    id: portSelector
    model: serialReader.getAvailablePorts()
    Layout.preferredWidth: 120

    // Refresh port list when clicked
    onPressedChanged: {
        if (pressed) {
            model = serialReader.getAvailablePorts();
        }
    }
}

Button {
    id: connectButton
    text: "Connect"
    Layout.preferredWidth: 80

    onClicked: {
        if (text === "Connect") {
            if (serialReader.connectToPort(portSelector.currentText)
                text = "Disconnect";
            } else {
                connectionErrorDialog.open();
            }
        } else {
            serialReader.disconnectFromPort();
            text = "Connect";
        }
    }
}

Item { Layout.fillWidth: true } // Spacer
```


Then adjust your `main.qml` layout to accommodate the new serial control panel:

```
Dashboard {
    id: dashboard
    anchors {
        top: serialControlPanel.bottom // Position under the serial panel
        left: parent.left
        right: parent.right
        bottom: controlsPanel.top
        margins: 10
    }
}
```

Option 2: Raspberry Pi Approach

If you're using a Raspberry Pi, you have additional options:

Method 1: Run the Qt Application on the Raspberry Pi

1. Cross-compile your Qt application for the Raspberry Pi
2. Connect the potentiometer directly to the Pi's GPIO pins
3. Modify your C++ code to read GPIO values directly

Method 2: Use Network Communication

1. Run a small server on the Raspberry Pi that reads the potentiometer values
2. Use WebSockets or TCP/IP to communicate between your Pi and PC
3. Update your Qt application to connect to this server

Raspberry Pi GPIO Code Example (Python)

python

```
import time
import board
import busio
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
import socketserver
import json

# Initialize the ADC (Analog-to-Digital Converter)
i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1115(i2c)

# Create analog input channels
speed_channel = AnalogIn(ads, ADS.P0)
fuel_channel = AnalogIn(ads, ADS.P1)
temp_channel = AnalogIn(ads, ADS.P2)
```

```
gear_channel = AnalogIn(ads, ADS.P3)

class DashboardTCPHandler(socketserver.BaseRequestHandler):
    def handle(self):
        while True:
            try:
                # Read potentiometer values and map them
                speed_value = map_value(speed_channel.value, 0, 26000, 0, 2)
                fuel_value = map_value(fuel_channel.value, 0, 26000, 0, 100)
                temp_value = map_value(temp_channel.value, 0, 26000, 50, 13)
                gear_value = map_value(gear_channel.value, 0, 26000, 1, 6)

                # Create JSON data
                data = {
                    "speed": speed_value,
                    "fuel": fuel_value,
                    "temperature": temp_value,
                    "gear": gear_value
                }

                # Send data
                self.request.sendall((json.dumps(data) + "\n").encode())
                time.sleep(0.1)
            except Exception as e:
                print(f"Connection closed: {e}")
                break

def map_value(x, in_min, in_max, out_min, out_max):
    return (x - in_min) * (out_max - out_min) // (in_max - in_min) + out_min

if __name__ == "__main__":
    HOST, PORT = "0.0.0.0", 9999 # Listen on all interfaces

    with socketserver.TCPServer((HOST, PORT), DashboardTCPHandler) as server:
        print(f"Server running on port {PORT}")
        server.serve_forever()
```

Then, you would create a corresponding network client in your Qt application to receive this data.

Adding the Required QML Components

For the Qt project, add these lines to your `.pro` file to include the serial port module:

```
QT += serialport
CONFIG += c++11
```

Project Summary

The complete system will have:

1. Hardware:

- Potentiometer(s) connected to Arduino or Raspberry Pi
- USB connection to your PC (or network for Raspberry Pi)

2. Embedded Code:

- Arduino sketch or Raspberry Pi program to read analog values
- Protocol to send values to PC (serial or network)

3. Desktop Application:

- SerialReader or NetworkClient class to receive values
- Integration with your existing DashboardModel
- UI to connect/disconnect from the hardware